

- 1. The concept of key stretching: A password like "password123" is easy for a human to remember, but it contains very little entropy. If we used the password directly as the key, an attacker could test millions of variations per second. Key stretching forces the computer to perform the hash function repeatedly before obtaining the final key. With -iter 1, the password passes through SHA-512 only once. It is an instantaneous operation (fractions of a microsecond). With -iter 100 000, the password is hashed, the result is re-hashed, and so on for 100 000 times.
- 2. The Impact on the Attacker (Work Asymmetry): The number of iterations creates a massive obstacle for anyone attempting a dictionary attack: For the legitimate user, waiting 0.5 seconds to derive the key and decrypt the file is not a problem. It happens only once. If the attacker has a dictionary of 1 million passwords and each attempt takes 0.5 seconds due to the high number of iterations, the attack will take 5.7 days instead of a few seconds. The higher the number of iterations, the more expensive (in terms of time and computing power) each individual attempt to guess the password becomes.
- 3. Analysis of the command in the tutorial: The -iter 1 setting is extremely insecure. With a single iteration, your "attacker" in the tutorial will be able to scan the entire dictionary almost instantly. Lack of resistance to brute-force attacks: There is no computational "brake." If the password is in the dictionary, it will be found in the blink of an eye.
- 4. Evolution of Standards: In the past, a few iterations were common. Today, guidelines recommend hundreds of thousands of iterations to counter the destructive power of modern GPUs and dedicated chips.

T3 - Tutorial: Practical dictionary attack to the symmetric encryption

Goal of this tutorial So, do not use a password that is in the dictionary of your language

The goal of this tutorial is to **mimic the behavior of an attacker who has intercepted an encrypted message and then tries to decrypt it by means of a dictionary attack.**

Assumptions and Definitions There are plenty of assumptions: in a real world, we need to do a lot of extra work that here is assumed

Let's suppose that **the attacker has been able to intercept and record a specific message encrypted using a well-known symmetric block cipher. We assume that the attacker knows the specific cipher used for encrypting the message and the different encryption parameters that have been selected by the sender. In practice, the whole security (i.e., confidentiality) of the transmission is based on a strong cipher and the confidentiality of the symmetric encryption key.** that is standard

The command used by the sender to encrypt the message is:

`openssl enc -e -aes-256-cbc -iter 1 -md sha512 -in plaintext.txt -out ciphertext.bin -pass pass:???????`
hashing the password passed by -pass

where "???????" is the secret "password" used to obtain the symmetric key used by the cipher. This specific operation for getting a key from a password is called "**derivation**". **How is this derivation performed?** In the command above, the secure hash function "sha512" is run a single time (as set by the "-iter" parameter). **How does the number of iterations used in this derivation affect the security of the message encryption? Consider very carefully this point.**

(assumption)
Somehow, the attacker knows in advance that the sender has made a very bad choice of the "password" used for encrypting the message. In fact, the "password" used is included in the Italian dictionary that is available in most GNU/Linux distributions. For this reason, the approach that will be followed by the attacker to break the message encryption is to try all the possible keys in this case (i.e., **all the words** that are in the Italian dictionary) **as fast as possible**. This operation must be done considering two important aspects:

- **avoid false negatives:** a specific word is tried but not detected as the correct key for decrypting the message; (attack failure)

You have the password and you use the secure hash function for obtaining a key that now can be used for the AES and then performing the encryption itself: key is a sequence of bits and the password is alphanumeric

1. Il "Generated Output" (L'output grezzo): È il risultato del calcolo dell'AES applicata con la password che hai scelto (quella del dizionario). Se la password è sbagliata, il generated output è un blocco di 16 byte di puro caos (es. A7 4F 99 21 ... 08). Questo output è quello che la funzione di decifrazione "sforna" prima di fare qualsiasi controllo di qualità.
2. L' "Expected Output" (L'aspettativa del Padding): OpenSSL è programmato per seguire una regola rigida (lo standard PKCS#7). Questa regola dice: "Ogni file cifrato deve finire con un codice che indica quanto spazio vuoto è stato aggiunto". L' Expected Output è la struttura che OpenSSL si aspetta di trovare analizzando l'ultimo byte del generated output: Se l'ultimo byte è 08, OpenSSL si aspetta che anche i 7 byte precedenti siano tutti 08. Se l'ultimo byte è 01, OpenSSL si aspetta che solo quell'ultimo byte sia di riempimento.

- minimize or avoid **false positives**: a word that is unable to decrypt the message into the correct plaintext is reported as correct.

Dictionary Attack

NOTE: all the scripts used in this tutorial are available on Virtuale.

The first attempt by the attacker is:

`./brute-force-openssl-1.sh ciphertext.bin`

It is clear that the result is very different from what we expected. In fact, OpenSSL has not emitted an error and it has reported as correct a message that is not in line with the baseline assumptions made in the first part of this tutorial. **Why does this happen?**

(Wrong assumption: OpenSSL is able to detect if the decryption has been made correctly or not)

The motivation is simple and it is about the semantics of errors reported by OpenSSL. **It has no way to detect if the used key is correct or not.** In fact, it returns an error only if there is an alignment problem of the generated output with respect to the expected output (i.e., different output sizes). In practice, this means that brute-forcing a message encrypted by using OpenSSL and following this approach would generate a relevant number of false positives. We can verify this by using the next version of the brute-forcing script:

so the attacker has to check by himself if the decryption is correct or not

generated output has different number of bits than the expected one

so, we can't stop when an output has no error (we need to check all the success ones for OpenSSL)

`./brute-force-openssl-2.sh ciphertext.bin`

How is it possible to mitigate the problem of false positives without introducing false negatives?

When you decrypt a file with the wrong password, the result is a bunch of random bytes. In computer science, a byte can represent 256 different values: Only a small portion corresponds to letters of the alphabet (a-z, A-Z). The vast majority correspond to non-printable characters (control codes, strange symbols, bytes that the computer doesn't know how to display). The attacker's assumption is: "If the message is real, it must contain words, so it must have 'x' consecutive normal letters one after another."

A straightforward possibility is to use a simple heuristic function that is based on yet another assumption: the message contains at least a given number of consecutive letters (vs. characters that are not printable or numbers). It should be clear that this heuristic is very simple and based on very strong assumptions that may not be realistic. In fact, for many messages that are transmitted it could fail. **Do you have in mind other possible solutions?**

`./brute-force-openssl-3.sh ciphertext.bin`

We can see that even using the proposed heuristic, we have been unable to avoid all the false positives even if we have been able to reduce them. What we can do is **speed up the execution** of the dictionary attack. This can be done quite easily by using the GNU parallel tool. In fact, we know that exploring a universe of passwords is an embarrassingly parallel problem. This means that we can parallelize the exploration of the Italian dictionary by using multiple "worker" programs running in parallel on different CPU cores. This operation is easy to

Why is it slow?
Because I'm using only one CPU.
Need to parallelize the script to get faster

implement but we have to slightly modify the architecture of our brute-forcing script. In fact, we need a “main” script that controls the execution of many “satellite” scripts.

Master (sequential bootstrap)

Workers (parallel execution)

WARNING: typically GNU parallel is not among the default packages installed in GNU/Linux distributions. You need to install it.

```
> ./brute-force-openssl-4-main.sh ciphertext.bin
```

There are some residual aspects that should be considered in a real-world implementation of the attack. For example,¹ is the OpenSSL execution in our computer able to exploit the hardware implementation of AES provided by most modern CPUs? This happens only if it is supported and enabled in: OpenSSL, the Linux kernel, and the CPU.² Another relevant aspect is the possible usage of a cluster for further parallelizing the exploration and the ability to run AES on GPUs.

You can spot it by easily see how fast is the execution: hardware execution is faster than the software one. It is no more possible to know if OpenSSL is able to use the hardware speed up or not (they removed the possibility to check it): by default, hardware acceleration (not possible to change in OpenSSL). It is possible to check if the Linux Kernel and CPU is using the hardware acceleration.